Rotom：定数乗算を用いたMCU向け高速暗号の設計

森田 健斗[1,a)]　峯松 一彦[2,6]　髙 和真[3,6]　阪本 光星[4,6]　上野 嶺[5]　五十部 孝典[6]

概要：マイクロコントローラ（MCU）では, 少ない実行サイクル, 小さな ROM/RAM 使用量, および定数時間実装を両立する暗号プリミティブが求められ，しばしば ARX ベース暗号がそのソフトウェア実装効率の高さから設計の選択肢として採用されてきた．しかしながら，ARX ベース暗号は暗号全体の差分・線形特性確率に対する上界を明示することが一般に容易ではない．これを踏まえて, 本稿ではソフトウェア実装効率と明示的な差分・線形特性の両立を目的とした MCU 向け Tweakable ブロック暗号 Rotom を提案する．Rotom では, 対象 MCU 上で低コストに実行できる定数乗算を非線形演算として採用し, 奇数定数による法 2^{16} 上の乗算と軽量な線形写像を交互に配置することで, 16 ビット S-box を構成する．結果として, Rotom は Tweak 機能と明示的な差分・線形特性を提供しながら, ARX ベース暗号と競争力のある性能を達成し, 最先端の AES 定数時間実装と比較して約 1/2 のサイクル数で動作する．

キーワード：Tweakable ブロック暗号, 差分・線形攻撃, 定数乗算，Cortex-M3/M4

Rotom : Design of a Fast MCU-oriented Cipher Using Constant Multiplication

Kento Morita[1,a)]　Kazuhiko Minematsu[2,6]　Kazuma Taka[3,6]　Kosei Sakamoto[4,6]　Rei Ueno[5]
Takanori Isobe[6]

Abstract: Microcontrollers (MCUs) require cryptographic primitives that simultaneously achieve low cycle counts, small ROM/RAM footprints, and constant-time implementations. ARX-based ciphers have often been adopted in such environments because of their high software efficiency. However, it is generally difficult to establish explicit upper bounds on the differential and linear characteristic probabilities of the full cipher. Motivated by this challenge, we propose Rotom, an MCU-oriented tweakable block cipher designed to combine software efficiency with explicit differential and linear characteristic bounds. Rotom employs constant multiplication as an efficient nonlinear operation on the target MCUs and constructs a 16-bit S-box by alternating multiplication by an odd constant modulo 2^{16} with lightweight linear mappings. Rotom provides native tweak support and explicit differential and linear characteristic bounds while achieving performance competitive with ARX-based ciphers. It operates in approximately half the number of cycles required by state-of-the-art constant-time AES implementations.

Keywords: Tweakable Block Cipher, Differential/Linear Attack, Constant Multiplication, Cortex-M3/M4

1. はじめに

1.1 背景

組込み機器や IoT デバイスでは，限られた計算資源と電力の下で暗号処理を実行する必要がある．特に，低消費電力広域ネットワークや IoT の発展に伴い，極めて低消費電力な端末における暗号処理の重要性が高まってい

[1] 兵庫県立大学 / University of Hyogo, Japan
[2] 日本電気株式会社 / NEC Corporation, Japan
[3] NICT / NICT, Japan
[4] 三菱電機株式会社 / Mitsubishi Electric Corporation, Japan
[5] 京都大学 / Kyoto University, Japan
[6] 大阪大学 / The University of Osaka, Japan
[a)] moritakent0515@gmail.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

る [1–4]．このような機器で用いられるマイクロコントローラ（MCU）では，安全性だけでなく，実行サイクル数，ROM/RAM 使用量，および定数時間実装の容易さが重要となる．中でも ARM Cortex-M3/M4 は，組込み・IoT 機器で広く利用される代表的な 32 ビット MCU である [5]．

Tweakable ブロック暗号（TBC）は，通常のブロック暗号に tweak を加えた暗号プリミティブである [6]．tweak にはアドレス，カウンタ，デバイス識別子，データ種別などを符号化でき，同一鍵の下で異なる処理領域や通信コンテキストを効率的に分離できる．また，TBC を用いた認証暗号やメッセージ認証コードには，誕生日限界を超える安全性と高い効率を両立する方式が知られている [7–9]．さらに，tweak を固定すれば通常のブロック暗号としても利用できるため，TBC は組込み・IoT 環境における多様な暗号方式の基盤となる汎用的なプリミティブである．

1.2 既存研究と課題

32 ビット MCU 向けソフトウェア暗号では，整数加算，回転，XOR を用いる ARX 型暗号が高い実装効率を示すことが知られている．なかでも Speck は，実行サイクル数，コードサイズ，RAM 使用量のいずれにおいても最高クラスであり，MCU 向け暗号の重要な性能目標となっている．一方，整数加算のキャリーにより差分・線形伝播が複雑になるため，多数ラウンドにわたる差分・線形特性確率の明示的な上界を導出することは容易ではない [10, 11]．解析性を高めた ARX 型暗号も提案されているが [12, 13]，追加構造に伴う実装コストが必要となる．

これに対し，S-box 型暗号では，S-box の差分分布表（DDT）および線形近似表（LAT）とアクティブ S-box 数の下界を組み合わせることで，暗号全体の差分・線形特性確率を明示的に評価できる．しかし，定数時間実装では通常の表参照を利用できず，bitslicing や fixslicing を用いた場合も [14, 15]，小型 MCU 上で Speck に匹敵する性能を得ることは容易ではない．

MSX は、MCU の多くでワード幅の整数乗算が 1 命令実行可能なことに着目し, これを利用することで，高いソフトウェア効率と差分・線形攻撃に対する強い安全性保証の両立を目指している [16]．しかし, S-box に相当する処理が鍵付きであり, 差分・線形攻撃以外の攻撃手法に対する安全性評価を精密に行うことが困難である．また多数のサブ鍵に伴うメモリアクセスのため，MCU 上の性能は Speck より大きく低下する．

したがって，**Speck に迫るソフトウェア実装効率，暗号全体の差分・線形特性確率に対する明示的な上界，および tweak 機能を同時に実現すること**が，本研究の設計上の課題である．

1.3 本研究の貢献

本稿では，上記の課題を解決する 128 ビット TBC 族 Rotom を提案する．Rotom の中心的なアイデアは，MCU 上で高速な整数乗算を用いて鍵なしの固定された S-box を実現することである．これにより，高いソフトウェア実装効率を維持しながら，暗号全体の差分・線形特性確率に対する明示的な上界を導出可能にする．Rotom の設計の特徴を以下に述べる．

(1) **定数乗算に基づく 16 ビット S-box．**法 2^{16} 上で置換となる奇数定数乗算と低コストな全単射線形写像を組み合わせ，Cortex-M3/M4 上で表参照なしに効率的に実装できる固定 16 ビット S-box を設計する．16 ビットのセルサイズは，乗算による高い非線形性と，DDT および LAT を全数評価できる解析コストを両立する．さらに，S-box を公開かつ鍵非依存とすることで，得られた局所的な差分・線形特性をすべての鍵に共通する暗号全体の安全性評価に利用できる．

(2) **ソフトウェア指向再帰型線形層と S-box の協調設計．**従来，主としてハードウェア向けに研究されてきた再帰型線形層 [17] を，32 ビット MCU 上で効率的に実行できるワード指向構造として新たに設計する．4 本の 32 ビットワードに対する in-place な XOR・回転演算のみで，高い 16 ビットセル単位の分岐数と低い実装コストを両立し，逆変換も同等のコストで実装できる．さらに，S-box の差分・線形特性，線形層のブランチ数，および実装コストを同時に考慮して両者を協調設計することで，暗号全体の差分・線形特性確率に対する明示的な上界と高い MCU 実装効率を両立する．

(3) **低オーバーヘッドな tweak 機能と算術加算 whitening．**tweakey 更新ではデータ処理部と同じ非線形層と線形層を再利用することで，ラウンド数を増やすことなく，関連 tweak 攻撃への耐性を得る．さらに，データ処理部の前後で whitening 鍵を 32 ビットワード単位の法加算により注入する．キャリーにより鍵依存性を非線形化することで，縮小ラウンド識別器から鍵回復攻撃への拡張を困難にする．

これら (1)–(3) の設計により，Rotom は明示的な安全性の根拠と高いソフトウェア実装効率を同時に達成する．結果として，Cortex-M3/M4 向け手書きアセンブリ実装では，tweak 機能と差分・線形特性確率に対する明示的な上界を備えながら Speck に迫る性能を達成し，MSX より大幅に高速に動作する．また，代表的な fixsliced AES 定数時間実装と比較して約半分のサイクル数で動作し，暗号化・復号のスタック使用量は約 40 バイトにとどまる．

2. Tweakable ブロック暗号 Rotom

Rotom は，128 ビット平文 P と 128 ビット tweak T を，

© 2026 Information Processing Society of Japan

$\kappa \in \{128, 256\}$ ビット鍵 K の下で，128 ビット暗号文 C に写像する TBC 族である．本暗号族は，128 ビット鍵を用いる Rotom-128 ，Rotom-128* と，256 ビット鍵を用いる Rotom-256 ，Rotom-256* の 4 方式から構成される．アスタリスク付きの方式は，対応するフルラウンド方式から 1 ラウンドを削減し，鍵当たりのデータ量を制限した方式である．

本章では，本暗号族の構成部品，暗号化処理の仕様および主張する安全性を定義する．完全な擬似コードおよびラウンド定数一覧は付録 A.1 に示す．

2.1 構成部品の仕様

Rotom は，非線形層 $\mathcal{S} : \{0,1\}^{128} \to \{0,1\}^{128}$ と線形層 $\mathcal{M} : \{0,1\}^{128} \to \{0,1\}^{128}$ から構成され，それぞれ 16 ビットセルと 32 ビットワードを単位として処理する．そこで，128 ビット状態 X を $\langle X \rangle_{32} = (X_0, X_1, X_2, X_3)$ および $\langle X \rangle_{16} = (x_0, \ldots, x_7)$ と表し，$X_i = x_{2i+1} \,\|\, x_{2i}$ $(0 \le i < 4)$ と定める．

非線形層 $\mathcal{S}$.

$\mathcal{S}$ は $\langle X \rangle_{16} = (x_0, \ldots, x_7)$ に対し，各セルに置換 S_{16} を並列適用する．$c =$ 0x809b，$M_c(x) = cx \bmod 2^{16}$ とし，S_{16} を

$$S_{16} = M_c \circ L_{\mathrm{rot}} \circ M_c \circ L_{\mathrm{sh}} \circ M_c$$

と定義する．ここで，線形写像 $L_{\mathrm{sh}}, L_{\mathrm{rot}}$ は

$$\begin{aligned} L_{\mathrm{sh}}(x) &= x \oplus (x \gg 8), \\ L_{\mathrm{rot}}(x) &= x \oplus (x \lll_{16} 4) \oplus (x \lll_{16} 12) \end{aligned}$$

であり，$\gg$ と $\lll_n$ はそれぞれ論理右シフトと n ビット左回転を表す．

線形層 $\mathcal{M}$.

$\mathcal{M}$ は 4 個の 32 ビットワード $\langle X \rangle_{32} = (X_0, X_1, X_2, X_3)$ に対し，3 段の in-place 処理を適用する．各段では，$i = 0, 1, 2, 3$ の順に

$$X_i \leftarrow X_i \oplus (X_{p_i} \lll_{32} k)$$

と更新し，右辺には各代入直前の値を用いる．各段の (p_0, p_1, p_2, p_3) と k は次の通りである：

段	(p_0, p_1, p_2, p_3)	k
1	$(3, 2, 1, 0)$	24
2	$(2, 3, 1, 0)$	0
3	$(2, 3, 0, 1)$	13

2.2 暗号化処理の仕様

Rotom の暗号化は，tweakey 更新，whitening 処理，データ処理からなる．全体構成を図 1 に示す．

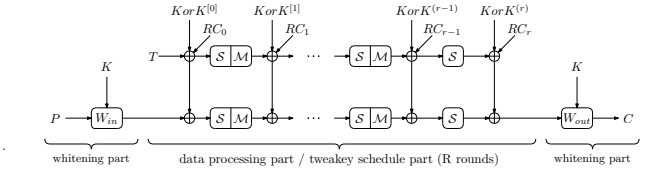


図 1 Rotom の暗号化処理.

ラウンド数を R とし，データ処理と tweakey 更新に共通するラウンド変換を

$$\mathcal{R}_r(Z) = \begin{cases} \mathcal{M}(\mathcal{S}(Z)), & 0 \le r < R-1, \\ \mathcal{S}(Z), & r = R-1 \end{cases}$$

と定義する．tweakey 状態は $U_0 = T$ とし，

$$U_{r+1} = \mathcal{R}_r(TK[r]) \qquad (0 \le r < R)$$

と更新する．また，データ状態 S_r を用いて暗号化を

$$\begin{aligned} S_0 &= \mathcal{W}_{\mathrm{in}}(P), \\ S_{r+1} &= \mathcal{R}_r\big(S_r \oplus TK[r]\big), \qquad 0 \le r < R, \\ C &= \mathcal{W}_{\mathrm{out}}\big(S_R \oplus TK[R]\big) \end{aligned}$$

と定める．以下では，鍵長ごとのラウンド tweakey $TK[r]$ と whitening 処理を定義する．

ラウンド定数は $\langle \mathrm{RC}_r \rangle_{32} = (C_r, 0, 0, 0)$ とし，C_r は π の 16 進小数部を 32 ビットごとに区切った第 r ワードである．具体値を付録 A.1.1 に示す．また，$\boxplus_{32\times 4}$ は，ワード間でキャリーを伝播しない 32 ビットワードごとの法 2^{32} 加算を表す．

128 ビット鍵方式.

Rotom-128 および Rotom-128* では, ラウンド tweakey を

$$TK[r] = U_r \oplus K \oplus \mathrm{RC}_r \qquad (0 \le r \le R)$$

とする．入力・出力 whitening は

$$\begin{aligned} \mathcal{W}_{\mathrm{in}}(Z) &= \mathcal{M}\left(Z \boxplus_{32\times 4} K\right), \\ \mathcal{W}_{\mathrm{out}}(Z) &= \mathcal{M}(Z) \boxplus_{32\times 4} \left(K \lll_{128} 64\right) \end{aligned}$$

と定義する．

256 ビット鍵方式.

Rotom-256 および Rotom-256* では，256 ビット鍵を $K = K^{[0]} \,\|\, K^{[1]}$ と二つの 128 ビット部分鍵に分割する．ラウンド r の部分鍵を

$$K^{(r)} = \begin{cases} K^{[0]}, & r \equiv 0 \pmod 2, \\ K^{[1]}, & r \equiv 1 \pmod 2 \end{cases} \qquad (0 \le r \le R)$$

とし，ラウンド tweakey を

$$TK[r] = U_r \oplus K^{(r)} \oplus \mathrm{RC}_r \qquad (0 \le r \le R)$$

とする．また，$a, b \in \{0, 1\}$ に対し，

© 2026 Information Processing Society of Japan

$$W_{2a+b} = \left(K^{[0]} \lll_{128} 32a\right) \boxplus_{32\times4} \left(K^{[1]} \lll_{128} 64b\right)$$

とし，入力・出力 whitening を

$$\mathcal{W}_{\text{in}}(Z) = \mathcal{M}\left(\mathcal{M}\left(Z \boxplus_{32\times4} W_0\right) \boxplus_{32\times4} W_1\right),$$
$$\mathcal{W}_{\text{out}}(Z) = \mathcal{M}\left(\mathcal{M}(Z) \boxplus_{32\times4} W_2\right) \boxplus_{32\times4} W_3$$

と定義する．

2.3 主張する安全性

QARMAv2 [18] と同様に，単純な攻撃の最適化や定数倍の改善に対する余裕として $\varepsilon = 16$ ビットを設け，鍵長 κ に対する時間計算量を $T = 2^{\kappa-\varepsilon}$ とする．各方式の鍵長 κ，ラウンド数 R，マスター鍵当たりの最大データ量 D，および主張する時間計算量 T を表 1 に示す．

表 1　Rotom のパラメータと主張する安全性．

方式	κ	R	D	T
Rotom-128	128	6	2^{128}	2^{112}
Rotom-128*	128	5	2^{80}	2^{112}
Rotom-256	256	7	2^{128}	2^{240}
Rotom-256*	256	6	2^{80}	2^{240}

いずれの方式についても，関連鍵，既知鍵，および選択鍵設定に対する安全性は主張しない．これは軽量暗号において一般的な仮定である [16, 18].

3. Tweakable ブロック暗号 Rotom の設計

本章では，Rotom の各構成要素の設計理由を述べる．Rotom は，**(i)ARM Cortex-M3/M4 上での高速かつ定数時間な実装**，**(ii) 暗号全体に対する明示的な差分・線形特性**，および **(iii)tweak 機能の両立**を設計目標とする．

3.1 定数乗算型 S-box の設計

Rotom の非線形層は，要件 (i),(ii) に適した部品として乗算に着目し，奇数定数を用いた整数乗算に基づく S-box を構成する．

既存の乗算利用の背景．

乗算を暗号の非線形演算として利用することは新しくなく，古くは IDEA が法 $2^{16}+1$ 上の乗算によって状態とサブ鍵を混合しており，以来，様々な構成が検討されてきた [19]．近年でも，MOE は奇数定数を用いた整数乗算を暗号部品として利用し [20]，MSX は整数乗算に基づく strongly universal hash をラウンド関数に用いている [16].

これらの採用理由の多くは，ソフトウェア上で乗算命令が，少ないサイクル数で実行可能であり，複雑なビット依存関係による強力な非線形性を導入できる点にある．特に，Cortex-M3/M4 上では，`MUL` 命令により，32 ビット整数同士の積の下位 32 ビットを，入力値によらず 1 サイクルで計算できる．そのため，表参照や分岐を用いない，定数時間実装という観点においても，乗算は MCU 上において有力な非線形演算である [16].

一方，既存方式における乗算の利用方法には，以下の三つの課題が生じる．第一に，乗数が鍵に依存する場合，その値によって乗算の暗号学的性質が変化し得る．実際，MOE の設計者らによる既存方式の整理によれば，IDEA や XMX のように鍵を乗数として状態と混合する方式の一部では，weak key が報告されている [20]．第二に，法の選択によっては乗算自体の実装コストが増加する．IDEA の法 $2^{16}+1$ 上の乗算や素数体上の乗算では，通常の整数乗算に加えて剰余処理が必要となり，MCU 上で追加のコストが生じる [16, 19]．第三に，MSX のように整数乗算を用いた鍵付き universal hash によって強い差分・線形保証を得る手法では，その構造を実現するために多数のサブ鍵を必要とし，サブ鍵の管理に伴う実装コストが生じる．MSX の設計者ら自身も，これらのサブ鍵に伴う load/store が MCU 上での性能低下の一因であると指摘している [16].

定数乗算型 S-box の選択．

そこで Rotom では，公開された固定定数 c による法 2^w 上の乗算

$$M_{c,w}(x) = cx \bmod 2^w$$

を基本非線形演算とする S-box を構築する．ここで，$M_{c,w}$ を w ビット幅の置換とするため，c を奇数に限定する．

まず，乗数を鍵から切り離し公開の固定定数とすることで，第一の課題である乗数の鍵依存性に起因する weak-key class を構成上排除できる．加えて，全ての鍵に対して同一の写像となることから，その差分・線形特性を固定 S-box として直接評価することが可能である．次に，法を 2^w とすることで，$M_{c,w}$ は積の下位 w ビットを取り出すことで計算できる．すなわち第二の課題について，剰余処理はビット幅の切り出しのみで済み，素数体上の乗算などに比べて追加の実装コストは小さい．さらに，第三の課題のように強い構造を構築するため多数のサブ鍵を扱う必要もない．

ただし，奇数定数乗算は低コストな置換である一方，単独では十分な差分・線形特性を持つ S-box とはならない．$M_{c,w}$ は下位ビットから上位ビットへの一方向の依存のみを持つ三角構造であり，

$$M_{c,w}(x \oplus 2^{w-1}) = M_{c,w}(x) \oplus 2^{w-1},$$
$$M_{c,w}(x) \bmod 2 = x \bmod 2$$

が成り立つ．すなわち，最上位ビットの差分は確率 1 で伝播し，最下位ビットには完全な線形関係が残る．しかも，奇数定数 a, b に対して

$$M_{b,w} \circ M_{a,w} = M_{ab \bmod 2^w, w}$$

が成り立つため，奇数定数乗算を単に繰り返しても一つの

© 2026 Information Processing Society of Japan

奇数定数乗算に帰着してしまい，この弱点は解消されない．

そこで，定数乗算の間に二進体上の可逆線形写像を挿入する．同様の考え方は MOE でも議論されており，定数乗算単体の弱点を補う手段として，二つの定数乗算の間に二進線形写像を挿入する S-box 構成が検討されている [20]．ただし，MOE が対象とする耐 Trojan 実装では，乗算と二進線形写像の間で sharing/recombination を要するため，その実装コストからこの構成は採用されていない．Rotom では，この考え方を MCU 向けソフトウェア実装に具体化し，回転，シフト，XOR のみからなる低コストな可逆線形写像を定数乗算の間に挿入する．線形写像は，確率 1 で伝播する上位ビット差分や完全な相関を持つ下位ビットマスクを別のビット位置へ移し，後続する乗算においてキャリーの影響を受けるようにする．これにより，定数乗算の低い実装コストを維持しつつ，三角構造に由来する弱い差分・線形特性を補う．

16 ビット S-box への具体化．

この構成のもとで，高い実装効率と良好な差分・線形特性を両立する S-box を探索する．

演算幅 w は，要件 (i) と (ii) の兼ね合いから $w = 16$ とする．8 ビット幅では 32 ビットレジスタ幅に対して，セルの抽出や再配置のコストが増加し，32 ビット幅では DDT および LAT の全数計算が困難となり，要件 (ii) を満たせないからである．16 ビット幅であれば，セルの抽出や再配置を低コストに実装でき，DDT および LAT も現実的な計算量で全数計算できる．

よって，探索空間は，すべての 16 ビット奇数定数と，Cortex-M3 上で 2–3 命令で実装可能な可逆線形写像の組合せである．各候補について最大差分確率 $p_{\max}$，最大二乗相関 $q_{\max}$，および実装コストを評価し，最良の S-box を探索する．

詳細な探索空間，探索方法および候補間の比較は付録 A.2.1 に示す．結果として，8 サイクルで実装可能であり，$p_{\max} = 2^{-9.61}$，$q_{\max} = 2^{-9.17}$ を達成する節 2.1 の S-box を採用する．

3.2 再帰型線形層の設計

Rotom の線形層では，非線形層 $\mathcal{S}$ が動作する 8 個の 16 ビットセル上で高い拡散性を持ち，かつ実装時の状態表現，すなわち 128 ビット状態を 4 個の 32 ビットワードに詰めた形式の上で効率的に動作することが求められる．

8 個の 16 ビットセル上の密な行列は，分岐数の指標に直接対応する．しかし，32 ビットワードに詰められた 16 ビットセルの抽出，位置合わせ，マスクおよび再パックを要するため，追加の命令数とレジスタ圧迫を生じる．そこで Rotom では，$\mathcal{M}$ を 4 個の 32 ビットワード上で直接構成する．

具体的には，ハードウェア向けに研究されてきた再帰型線形層 [21, 22] の考え方をソフトウェア向けに適用する．すなわち，セル単位の密な行列を評価する代わりに，

$$X_d \leftarrow X_d \oplus (X_s \lll_{32} k), \qquad d \neq s$$

という低コストなワード更新を繰り返し適用する．更新されたワードを後続の更新の入力として再利用できるため，少数の基本演算から再帰的な拡散が得られる．

この構成は実装面と解析面の双方で利点を持つ．実装面では，回転付き XOR を Cortex-M3/M4 のバレルシフタにより低コストに実装でき，状態を 4 本のレジスタに保ったまま更新できる．また各基本演算は自己逆であるため，線形層全体の逆演算は同じ更新を逆順に適用するだけで得られ，逆演算も同等のコストで実現できる．解析面では，各基本演算を 32 ビットワード上の線形写像として表し，その合成を 16 ビットセル上の線形写像へ変換することで，実装を 32 ビットワード単位に保ったまま 16 ビットセル単位の分岐数を評価できる．

詳細な探索空間，探索方法および候補間の比較は付録 A.2.3 に示す．結果として節 2.1 に示す，分岐数 7 の線形層を採用する．

3.3 tweakey 更新の設計

tweakey 状態を線形に更新すると，TBC において強力な関連 tweak 差分攻撃において，差分が非線形部品を活性化せずに伝播し得る．そこで，tweakey 更新にもデータ処理と同じ $\mathcal{S}$ および $\mathcal{M}$ を用いる．これにより，関連 tweak 差分攻撃においても差分が暗号化処理部分と同様に非線形部品を活性化し，弱い遷移を許容しない．また，データ処理部と同様の部品を用いることでコードサイズを抑えた実装が可能となる．

3.4 whitening 処理の設計

whitening 処理では，32 ビットワードごとの法加算と線形層 $\mathcal{M}$ を組み合わせることで鍵回復ラウンドに対するラウンド追加と同様の効果を低コストで実現する．XOR-whitening では，線形層を等価な XOR 部分鍵へ吸収できるが，法加算ではキャリーが平文と鍵の双方に依存し，直後の $\mathcal{M}$ によって状態全体へ拡散される．そのため，削減ラウンド識別器の外側で推測しなければならない鍵ビット数が増加し，標準的な鍵回復拡張を困難にする．この whitening 処理は，各鍵加算で 4 回の 32 ビット加算のみを要し，追加の非線形層を置く場合の 8 個の S-box 評価より低コストである．

4. 安全性評価

本章では，Rotom の差分攻撃，線形攻撃および鍵回復攻撃に対する安全性評価について述べる．

© 2026 Information Processing Society of Japan

4.1 差分・線形攻撃

S_{16} の DDT および LAT を全数計算した結果，最大差分確率 p_S および最大線形確率 q_S は

$$p_S = 84 \cdot 2^{-16} \approx 2^{-9.61}, \qquad q_S = \left(2728 \cdot 2^{-16}\right)^2 \approx 2^{-9.17}$$

であった．したがって，A 個のアクティブ S-box を含む差分特性確率および線形特性確率は，それぞれ高々 p_S^A および q_S^A である．

表 2 に，各ラウンド数におけるアクティブ S-box 数の下界を示す．この下界は，単一 tweak 差分，関連 tweak 差分，および単一 tweak 線形特性について共通である．関連 tweak 差分では，データ処理部と tweakey 更新部の双方に現れるアクティブ S-box を数える．

表 2 各ラウンド数におけるアクティブ S-box 数の下界．

ラウンド数	1	2	3	4	5	6
アクティブ S-box 数	1	7	9	14	17	21

表 2 より，3 ラウンドにおけるアクティブ S-box 数の下界は 9 である．すなわち最大差分特性確率および最大線形特性確率は，それぞれ $2^{-86.47}$ および $2^{-82.55}$ 以下となり，いずれの識別器を利用するにもデータ量を 2^{80} 個より多く用意する必要がある．また，4 ラウンドにおけるアクティブ S-box 数の下界は 14 である．すなわち最大差分特性確率および最大線形特性確率は，$2^{-134.51}$ および $2^{-128.42}$ 以下となり，いずれの識別器を利用するにもデータ量を 2^{128} 個より多く用意する必要がある．これらは表 1 に示すマスター鍵当たりの最大データ量より多い．そのため，主張する安全性の範囲内においてこれらの識別器を有効に活用することはできないと結論付けられる．以上より，Rotom は単一 tweak および関連 tweak 差分特性，ならびに単一 tweak 線形特性に基づく識別攻撃に対して，十分な耐性を有すると期待できる．

4.2 鍵回復攻撃

前節で得られた差分・線形特性の重みと，各方式の時間・データ計算量の上限を比較すると，攻撃に利用可能な識別器は，データ制限付き方式では最大 2 ラウンド，その他の方式では最大 3 ラウンドである．

一般に，このような識別器は，その前後のラウンドを部分暗号化または部分復号することで，鍵回復攻撃へ拡張される．しかし，Rotom の入出力境界では，32 ビットワードごとの法加算による whitening と線形層 $\mathcal{M}$ が連続して適用される．法加算によって生じるキャリー依存項は $\mathcal{M}$ によって状態全体へ拡散されるため，通常の S-box 型暗号のように，少数の独立な部分鍵のみを推測して外側ラウンドを処理することは困難である．

したがって，評価した識別器に外側ラウンドを追加する具体的な鍵回復攻撃は確認されていない．表 3 に，識別器が覆う最大ラウンド数と，各方式の総ラウンド数との差を示す．鍵回復攻撃では，この差に加え，whitening 処理での鍵加算においても攻撃者は鍵を推測する必要がある．

表 3 差分・線形攻撃に対する安全性マージン．

方式	識別器の最大ラウンド数	安全性マージン
Rotom-128*	2	3
Rotom-128	3	3
Rotom-256*	2	4
Rotom-256	3	4

5. ソフトウェア実装評価

本節では，ARM Cortex-M3 および Cortex-M4 上における Rotom のソフトウェア実装性能を評価する．まず，比較対象，評価環境，評価方法およびソフトウェア性能について示した後，サイドチャネル攻撃に対する考察を述べる．

5.1 評価条件

比較対象．

比較対象には，ISO 標準の ARX 型ブロック暗号 Speck と，乗算を用いた差分・線形に対する強い安全性保証を持つ MSX, NIST 標準暗号 AES の定数時間 fixslicing 実装を用いる．これらは Tweakable ブロック暗号ではないが，Rotom が tweak 機能を提供しながら，既存の高速なブロック暗号と競争力のある性能を達成することを示すための基準として採用する．

評価環境．

評価は，ARM Cortex-M3 および Cortex-M4 上で行う．具体的には以下の二つの評価ボードを用い，開発環境には STM32CubeIDE[*1]を用いる．

- STMicro NUCLEO F103RB [*2]
- STMicro NUCLEO F411RE [*3]

Rotom は，メモリアクセスおよび 16 ビットセルと 32 ビットワード間の packing 処理を最適化するため，Cortex-M3 および Cortex-M4 向けの手書きアセンブリとして実装する．Speck と MSX では先行研究 [16] で用いられた C 言語実装を用い，Rotom と同一の環境で測定する．AES については，先行研究 [15] で報告された 2 ブロック並列の fully-fixsliced アセンブリ実装の評価値を引用する．

C 実装のコンパイルには `arm-none-eabi-gcc` （GNU Tools for STM32 13.3.rel1，GCC 13.3.1）を使用し，最適化オプションは`-O3` を用いる．

[*1] https://www.st.com/en/development-tools/stm32cubeide.html
[*2] https://www.st.com/en/evaluation-tools/nucleo-f103rb.html
[*3] https://www.st.com/en/evaluation-tools/nucleo-f411re.html

© 2026 Information Processing Society of Japan

表 4　Rotom，Speck，MSX および AES の ARM Cortex-M 上での fully-unrolled 実装結果．I/O 欄の括弧内は，事前計算済みラウンド鍵またはラウンド tweakey などの追加入力を表す．AES の ROM 欄は Code（+Data）の順に示す．“–” は参照文献で値が報告されていないことを表す．

方式	処理	Blocks	Cortex-M3				Cortex-M4			
			Time	ROM	RAM		Time	ROM	RAM	
					I/O	Stack			I/O	Stack
128 ビット鍵方式										
Rotom-128*	tweakey 生成	–	530	1819	128	36	490	1699	128	36
	暗号化／復号	1	559/568	1885	16(+112)	40	519/528	1765	16(+112)	40
Rotom-128	tweakey 生成	–	625	2175	144	36	577	2031	144	36
	暗号化／復号	1	651/663	2233	16(+128)	40	603/615	2089	16(+128)	40
Speck-128/128	鍵生成	–	530	1479	272	16	523	1387	272	12
	暗号化／復号	1	635/648	1795	16(+256)	28	623/631	1811	16(+256)	12
MSX-128-128	鍵生成	–	706	1576	880	100	660	1488	880	104
	暗号化／復号	1	1034/1043	2620	16(+864)	44	973/962	2596	16(+864)	36
AES-128	鍵生成	1	1178	3848(+1024)	368	196	1255	3848(+1024)	368	196
	暗号化／復号	2	1273/–	9184	32(+352)	112	1275/–	9184	32(+352)	112
256 ビット鍵方式										
Rotom-256*	tweakey 生成	–	677	2247	160	40	629	2103	160	40
	暗号化／復号	1	725/745	2461	16(+144)	40	677/697	2317	16(+144)	40
Rotom-256	tweakey 生成	–	782	2615	176	40	726	2447	176	40
	暗号化／復号	1	818/839	2809	16(+160)	40	762/783	2641	16(+160)	40
Speck-128/256	鍵生成	–	578	1711	304	32	569	1623	304	28
	暗号化／復号	1	671/684	1907	16(+272)	28	659/667	1923	16(+272)	12
MSX-128-256	鍵生成	–	714	1668	896	108	678	1620	896	108
	暗号化／復号	1	1034/1043	2620	16(+864)	44	973/966	2596	16(+864)	36
AES-256	暗号化／復号	2	1745/–	12656	32(+480)	112	1747/–	12656	32(+480)	112

評価方法．

FELICS ベンチマークフレームワーク [23,24] の Scenario 0 に従い，128 ビットブロック当たりの実行サイクル数，ROM サイズおよび RAM 使用量を測定する．

tweakey 生成およびラウンド鍵生成は個別に測定し，暗号化および復号の測定では，事前計算ラウンド鍵設定および事前計算 tweak 鍵設定で測定する．

各方式のピーク性能を比較するため，ラウンド処理を完全に展開した fully-unrolled 実装の結果を示す．

5.2　評価結果

表 4 に評価結果を示す．RAM 使用量は，入出力ブロックおよび事前計算値を保持する I/O 領域と，実行時に使用する Stack 領域に分けて示す．

実行サイクル数．

128 ビット鍵方式では，Rotom-128* は Cortex-M3 および Cortex-M4 上で 559/568cycle および 519/528cycle を示し，全比較対象の中で最速である．Rotom-128 は Speck-128/128 と同等の性能を示し，256 ビット鍵方式では Rotom-256* が Speck-128/256 に近い性能を示す．また，乗算を用いる MCU 向けブロック暗号 MSX と比較すると，全ての Rotom 方式で暗号化サイクル数を削減している．

既報の fully-fixsliced AES 実装と比較しても，全ての Rotom 方式が高速である．128 ビット鍵方式では，Rotom-128* は Cortex-M3/M4 上でそれぞれ約 2.28 倍/2.46 倍，Rotom-128 は約 1.96 倍/2.11 倍高速である．256 ビット鍵方式では，Rotom-256* は約 2.41 倍/2.58 倍，Rotom-256 は約 2.13 倍/2.29 倍高速である．

RAM 使用量．

Rotom の事前計算済みラウンド tweakey に必要な I/O 領域は，対応する Speck，MSX および AES のラウンド鍵領域より小さい．特に MSX では 864 バイトの事前計算値を必要とするのに対し，Rotom では 112–160 バイトに収まる．また，暗号化および復号の Stack 使用量は全方式で 40 バイトであり，小さい RAM 使用量を維持している．

5.3　サイドチャネル攻撃に関する考察

Rotom は，乗算，加算，回転およびシフト命令で構成でき，ARM Cortex-M3 および Cortex-M4 上では，定数時間実装が可能である．

電力・電磁波解析への対策としては，A2B および B2A 変換 [25] を用いたマスキングを適用できる．これにより，

© 2026 Information Processing Society of Japan

S-box は非線形ガジェットを用いずにマスキング可能である．マスキング実装の詳細な評価は今後の課題とする．

6. まとめ

本稿では，ARM Cortex-M3/M4 上での高速かつ定数時間な実装と，暗号全体に対する明示的な差分・線形特性の評価を両立する TBC 族 Rotom を提案した．Rotom は，奇数定数乗算と軽量な可逆線形写像からなる 16 ビット S-box と，32 ビットワード上で動作する再帰型線形層を組み合わせ，実装コストと差分・線形特性を協調評価することで構成を決定した．さらに，非線形な tweakey 更新と法加算を用いた whitening 処理により，低コストに tweak 機能と鍵回復攻撃への耐性を付加した．

実装評価の結果，Rotom は Cortex-M3/M4 上で Speck に近い実行性能を示し，同じく乗算を用いる MCU 向け暗号 MSX より高速に動作した．また，定数時間 fixslicing による AES 実装と比較しても少ないサイクル数を実現した．

謝辞

本研究は, JST CRONOS JPMJCS25N1 および JSPS 科研費 JP24H00696 の支援を受けたものである．

参考文献

[1] Raza, U., Kulkarni, P. and Sooriyabandara, M.: Low Power Wide Area Networks: An Overview, *IEEE Communications Surveys & Tutorials*, Vol. 19, No. 2, pp. 855–873 (2017).

[2] Hoeller, A., Sant'Ana, J., Markkula, J., Mikhaylov, K., Souza, R. D. and Alves, H.: Beyond 5G Low-Power Wide-Area Networks: A LoRaWAN Suitability Study, *Sensors*, Vol. 20, No. 8, p. 2254 (2020).

[3] Butt, M. M., Mangalvedhe, N. R., Pratas, N. K., Harrebek, J., Kimionis, J., Tayyab, M., Barbu, O.-E., Ratasuk, R. and Vejlgaard, B.: Ambient IoT: A Missing Link in 3GPP IoT Devices Landscape, arXiv:2312.06569 (2023).

[4] Zheng, K., Xu, R., Mei, J., Yang, H., Lei, L. and Wang, X.: Ambient IoT towards 6G: Standardization, Potentials, and Challenges, arXiv:2412.12519 (2024).

[5] : 32 Bit MCU Market Overview, https://www.futuremarketreport.com/industry-report/32-bit-mcu-market.

[6] Liskov, M., Rivest, R. L. and Wagner, D.: Tweakable Block Ciphers, *CRYPTO 2002*, pp. 31–46 (2002).

[7] Jean, J., Nikolic, I., Peyrin, T. and Seurin, Y.: The Deoxys AEAD Family, *J. Cryptol.*, Vol. 34, No. 3, p. 31 (2021).

[8] Krovetz, T. and Rogaway, P.: The Software Performance of Authenticated-Encryption Modes, *FSE 2011*, pp. 306–327 (2011).

[9] Iwata, T., Minematsu, K., Peyrin, T. and Seurin, Y.: ZMAC: A Fast Tweakable Block Cipher Mode for Highly Secure Message Authentication, *CRYPTO 2017, Part III*, pp. 34–65 (2017).

[10] Fu, K., Wang, M., Guo, Y., Sun, S. and Hu, L.: MILP-Based Automatic Search Algorithms for Differential and Linear Trails for Speck, *FSE 2016*, pp. 268–288 (2016).

[11] Kurahara, R., Sakamoto, K. and Isobe, T.: Finding New Differential Characteristics on SPECK Family, *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, Vol. 108, No. 5, pp. 760–763 (2025).

[12] Dinu, D., Perrin, L., Udovenko, A., Velichkov, V., Großschädl, J. and Biryukov, A.: Design Strategies for ARX with Provable Bounds: Sparx and LAX, *ASIACRYPT 2016, Part I*, pp. 484–513 (2016).

[13] Beierle, C., Biryukov, A., dos Santos, L. C., Großschädl, J., Perrin, L., Udovenko, A., Velichkov, V. and Wang, Q.: Alzette: A 64-Bit ARX-box - (Feat. CRAX and TRAX), *CRYPTO 2020, Part III*, pp. 419–448 (2020).

[14] Adomnicai, A., Najm, Z. and Peyrin, T.: Fixslicing: A New GIFT Representation, *IACR Transactions on Cryptographic Hardware and Embedded Systems*, Vol. 2020, No. 3, pp. 402–427 (2020).

[15] Adomnicai, A. and Peyrin, T.: Fixslicing AES-like Ciphers, *IACR Transactions on Cryptographic Hardware and Embedded Systems*, Vol. 2021, No. 1, pp. 402–425 (2021).

[16] Minematsu, K., Suzaki, T., Rahman, M., Suryawanshi, S. and Isobe, T.: MSX: Lightweight Block Ciphers for Microcontrollers with High-Assurance against Differential and Linear Attacks, *IACR Communications in Cryptology*, Vol. 2, No. 4, pp. 1–32 (2025).

[17] Guo, J., Peyrin, T. and Poschmann, A.: The PHOTON Family of Lightweight Hash Functions, *CRYPTO 2011*, pp. 222–239 (2011).

[18] Avanzi, R., Banik, S., Dunkelman, O., Eichlseder, M., Ghosh, S., Nageler, M. and Regazzoni, F.: The QARMAv2 Family of Tweakable Block Ciphers, *IACR Transactions on Symmetric Cryptology*, Vol. 2023, No. 3, pp. 25–73 (2023).

[19] Lai, X. and Massey, J. L.: A Proposal for a New Block Encryption Standard, *EUROCRYPT 1990*, pp. 389–404 (1991).

[20] Bronchain, O., Faust, S., Lallemand, V., Leander, G., Perrin, L. and Standaert, F.-X.: MOE: Multiplication Operated Encryption with Trojan Resilience, *IACR Transactions on Symmetric Cryptology*, Vol. 2021, No. 1, pp. 78–129 (2021).

[21] Sajadieh, M., Dakhilalian, M., Mala, H. and Sepehrdad, P.: Recursive Diffusion Layers for Block Ciphers and Hash Functions, *FSE 2012*, pp. 385–401 (2012).

[22] Augot, D. and Finiasz, M.: Direct Construction of Recursive MDS Diffusion Layers Using Shortened BCH Codes, *FSE 2014*, pp. 3–17 (2014).

[23] Dinu, D., Corre, Y. L., Khovratovich, D., Perrin, L., Großschädl, J. and Biryukov, A.: FELICS - Fair Evaluation of LIghtweight Cryptographic Systems, `https://www.cryptolux.org/index.php/FELICS` (2015).

[24] Dinu, D., Corre, Y. L., Khovratovich, D., Perrin, L., Großschädl, J. and Biryukov, A.: Triathlon of Lightweight Block Ciphers for the Internet of Things, *Journal of Cryptographic Engineering*, Vol. 9, No. 3, pp. 283–302 (2019).

[25] Coron, J.-S.: High-Order Conversion from Boolean to Arithmetic Masking, *CHES 2017*, pp. 93–114 (2017).

[26] Beierle, C., Hebborn, P., Leander, G. and Perehuda, Y.: Integral Resistance of Block Ciphers with Key Whitening by Modular Addition, *CRYPTO 2025, Part V*, pp. 497–529 (2025).

© 2026 Information Processing Society of Japan

付　　録

A.1　詳細なアルゴリズム

Algorithm 1 Rotom の暗号化.

function ENCRYPT(P, T, K)
  ▶ ラウンド tweakey 生成
  $U \leftarrow T$
  for $r \leftarrow 0$ **to** $R-1$ **do**
    $TK[r] \leftarrow U \oplus K^{(r)} \oplus \mathrm{RC}_r$
    $U \leftarrow \mathcal{S}(TK[r])$
    if $r < R-1$ **then**
      $U \leftarrow \mathcal{M}(U)$
    end if
  end for
  $TK[R] \leftarrow U \oplus K^{(R)} \oplus \mathrm{RC}_R$
  ▶ 入力ホワイトニング
  if $\kappa = 128$ **then**
    $X \leftarrow P \boxplus_{32\times 4} K$
    $X \leftarrow \mathcal{M}(X)$
  else
    $X \leftarrow P \boxplus_{32\times 4} W_0$
    $X \leftarrow \mathcal{M}(X)$
    $X \leftarrow X \boxplus_{32\times 4} W_1$
    $X \leftarrow \mathcal{M}(X)$
  end if
  ▶ データ処理
  for $r \leftarrow 0$ **to** $R-1$ **do**
    $X \leftarrow X \oplus TK[r]$
    $X \leftarrow \mathcal{S}(X)$
    if $r < R-1$ **then**
      $X \leftarrow \mathcal{M}(X)$
    end if
  end for
  $X \leftarrow X \oplus TK[R]$
  ▶ 出力ホワイトニング
  if $\kappa = 128$ **then**
    $X \leftarrow \mathcal{M}(X)$
    $C \leftarrow X \boxplus_{32\times 4} (K \lll_{128} 64)$
  else
    $X \leftarrow \mathcal{M}(X)$
    $X \leftarrow X \boxplus_{32\times 4} W_2$
    $X \leftarrow \mathcal{M}(X)$
    $C \leftarrow X \boxplus_{32\times 4} W_3$
  end if
  return C
end function

Algorithm 2 Rotom の復号.

function DECRYPT(C, T, K)
  ▶ ラウンド tweakey 生成
  $U \leftarrow T$
  for $r \leftarrow 0$ **to** $R-1$ **do**
    $TK[r] \leftarrow U \oplus K^{(r)} \oplus \mathrm{RC}_r$
    $U \leftarrow \mathcal{S}(TK[r])$
    if $r < R-1$ **then**
      $U \leftarrow \mathcal{M}(U)$
    end if
  end for
  $TK[R] \leftarrow U \oplus K^{(R)} \oplus \mathrm{RC}_R$
  ▶ 出力ホワイトニングの逆変換
  if $\kappa = 128$ **then**
    $X \leftarrow C \boxminus_{32\times 4} (K \lll_{128} 64)$
    $X \leftarrow \mathcal{M}^{-1}(X)$
  else
    $X \leftarrow C \boxminus_{32\times 4} W_3$
    $X \leftarrow \mathcal{M}^{-1}(X)$
    $X \leftarrow X \boxminus_{32\times 4} W_2$
    $X \leftarrow \mathcal{M}^{-1}(X)$
  end if
  ▶ データ処理の逆変換
  $X \leftarrow X \oplus TK[R]$
  for $r \leftarrow R-1$ **downto** 0 **do**
    if $r < R-1$ **then**
      $X \leftarrow \mathcal{M}^{-1}(X)$
    end if
    $X \leftarrow \mathcal{S}^{-1}(X)$
    $X \leftarrow X \oplus TK[r]$
  end for
  ▶ 入力ホワイトニングの逆変換
  if $\kappa = 128$ **then**
    $X \leftarrow \mathcal{M}^{-1}(X)$
    $P \leftarrow X \boxminus_{32\times 4} K$
  else
    $X \leftarrow \mathcal{M}^{-1}(X)$
    $X \leftarrow X \boxminus_{32\times 4} W_1$
    $X \leftarrow \mathcal{M}^{-1}(X)$
    $P \leftarrow X \boxminus_{32\times 4} W_0$
  end if
  return P
end function

本付録では, Rotom の暗号化および復号の完全な擬似コードを示す．選択した方式に応じて, 鍵長 $\kappa \in \{128, 256\}$ とラウンド数 R は固定される．128 ビット鍵方式では $K^{(r)} = K$ とし，256 ビット鍵方式では $K = K^{[0]} \parallel K^{[1]}$ と分割して $K^{(r)} = K^{[r \bmod 2]}$ とする．また，256 ビット鍵方式で用い

© 2026 Information Processing Society of Japan

る $W_0, \ldots, W_3$ は本文の定義に従う．復号で用いる $\boxminus_{32\times4}$ は，4 個の 32 ビットワードごとの法 2^{32} 減算を表す．また，$\mathcal{S}^{-1}$ は各 16 ビットセルに S_{16}^{-1} を並列適用する写像であり，$c^{-1} = \texttt{0x6593}$ として

$$
\begin{aligned}
&x \leftarrow c^{-1}x \bmod 2^{16},\\
&x \leftarrow x \oplus (x \lll_{16} 4) \oplus (x \lll_{16} 12),\\
&x \leftarrow c^{-1}x \bmod 2^{16},\\
&x \leftarrow x \oplus (x \gg 8),\\
&x \leftarrow c^{-1}x \bmod 2^{16}
\end{aligned}
$$

を上から順に適用して得られる．$\mathcal{M}^{-1}$ は，$\mathcal{M}$ の 3 段を逆順にたどり，各段の逐次更新を $i = 3, 2, 1, 0$ の順に適用することで得られる．

A.1.1 ラウンド定数

ラウンド定数は $\langle \mathrm{RC}_r \rangle_{32} = (C_r, 0, 0, 0)$ であり，各 C_r を表 A·1 に示す．

表 **A·1** ラウンド定数を定める 32 ビット値 C_r.

r	C_r	r	C_r
0	0x243f6a88	4	0xa4093822
1	0x85a308d3	5	0x299f31d0
2	0x13198a2e	6	0x082efa98
3	0x03707344	7	0xec4e6c89

A.2 定数乗算型 S-box と再帰型線形層設計の詳細

本付録では，節 3.1 および節 3.2 で概要を示した，S-box と再帰的線形層の設計を詳述する．

暗号全体の実装効率は，S-box と線形層をそれぞれ独立に最適化するだけでは決まらない．S-box の局所的な強度と実装コスト，線形層の分岐数と実装コスト，およびそれらから決まる必要ラウンド数を総合的に評価する必要がある．

そこで，本設計では S-box と線形層を段階的に探索する．まず，定数乗算型 S-box の探索空間を定義し，推定実装コストごとに差分・線形特性の最も強い候補を選択する．次に，各 S-box 候補について，線形層の分岐数から所望の安全性を満たすための必要ラウンド数を見積もり，暗号全体の実装コストの観点から候補を絞り込む．最後に，再帰型線形層を探索して実現可能な分岐数と実装コストを求め，S-box との各組合せについて暗号全体の推定サイクル数を比較し，最終的なパラメータを選択する．

A.2.1 定数乗算型 S-box の探索

探索空間

奇数定数 c に対して，

$$M_c(x) = cx \bmod 2^{16}$$

とする．探索対象とする S-box は，次の 2 構造である．

$$
\begin{aligned}
\text{MLM}: &\quad M_c \circ L_1 \circ M_c,\\
\text{MLMLM}: &\quad M_c \circ L_2 \circ M_c \circ L_1 \circ M_c.
\end{aligned}
$$

各候補ではすべての乗算に同じ定数 c を用い，$c \in \{1, 3, \ldots, 2^{16}-1\}$ の全 2^{15} 通りを列挙する．MLMLM では，(L_1, L_2) を順序付き組として探索する．

内部線形写像 L_i には，Cortex-M 上で低コストに実装可能であり，16 ビット上で可逆かつ逆変換も同等のコストで実装できる写像を用いる．探索対象とした 5 種類の写像族を以下に示す．

$$
\begin{aligned}
\mathrm{ROT}_r(x) &= x \lll_{16} r,\\
\mathrm{SRxor}_s(x) &= x \oplus (x \gg s),\\
\mathrm{2ROTxor}_r(x) &= x \oplus (x \lll_{16} r) \oplus (x \lll_{16} (r+8)),\\
\mathrm{2SRxor}_{s,s'}(x) &= x \oplus (x \gg s) \oplus (x \gg s'),\\
\mathrm{2SALTxor}_{s,t}(x) &= x \oplus (x \ll s) \oplus (x \gg t).
\end{aligned}
$$

各写像族の推定サイクル数，パラメータ範囲，および候補数を表 A·2 に示す．推定サイクル数には，32 ビットレジスタ上で 16 ビット値を扱うための前処理 1 命令を含む．また，パラメータ範囲は，順変換と逆変換を同一の推定サイクル数で実装できる範囲とする．

表 **A·2** S-box 内部線形写像の候補族.

候補族	推定サイクル数	パラメータ	候補数
ROT	2	$0 \le r < 16$	16
SRxor	2	$8 \le s < 16$	8
2ROTxor	3	$0 \le r < 16$	16
2SRxor	3	$8 \le s, s' < 16$	64
2SALTxor	3	$8 \le s, t < 16,\ s \ne t$	112

コスト別の S-box 探索

奇数定数乗算を 1 サイクルと仮定し，内部線形写像の推定サイクル数と合わせて，S-box を推定サイクル数 τ ごとに分類する．2 サイクルの内部線形写像は 24 通り，3 サイクルの内部線形写像は 192 通りであり，MLM および MLMLM から表 A·3 に示す 5 つのコスト級を得る．

表 **A·3** 乗算–線形型 S-box の推定サイクル数と探索候補数.

構造	L_i のサイクル数	S-box サイクル数 τ	候補数
MLM	2	4	786,432
MLM	3	5	6,291,456
MLMLM	$2+2$	7	18,874,368
MLMLM	$2+3, 3+2$	8	301,989,888
MLMLM	$3+3$	9	1,207,959,552

各候補 S について，最大差分確率 $p_{\max}(S)$ および最大二

© 2026 Information Processing Society of Japan

乗相関 $q_{\max}(S)$ から

$$w_D(S) = -\log_2 p_{\max}(S),$$
$$w_L(S) = -\log_2 q_{\max}(S),$$
$$w(S) = \min\{w_D(S), w_L(S)\}$$

を定義し，$w(S)$ を評価値とする．サイクル数 τ の候補集合を C_τ とし，各コスト級について

$$S_\tau^\star = \arg\max_{S \in C_\tau} w(S)$$

を満たす候補を，枝刈りを用いて探索した．

さらに，各最良候補について，128 ビットの特性重みに必要なアクティブ S-box 数を

$$N_\tau = \left\lceil \frac{128}{w(S_\tau^\star)} \right\rceil$$

とする．探索結果を表 A·4 に示す．

表 **A·4** 各コスト級の最良 S-box 候補．小数第 3 位以下切り捨て．

τ	$w(S)$	N_τ	c	L_1	L_2
4	4.10	32	`0x7a5b`	SRxor_{10}	–
5	5.35	24	`0x62ad`	$\mathrm{2ROTxor}_3$	–
7	7.92	17	`0x2167`	SRxor_9	SRxor_{10}
8	9.17	14	`0x809b`	SRxor_8	$\mathrm{2ROTxor}_4$
9	10.30	13	`0xf693`	$\mathrm{2ROTxor}_3$	$\mathrm{2ROTxor}_3$

A.2.2 分岐数に基づく S-box 候補の絞り込み

前節で得た各 S-box 候補について，16 ビットセル単位の分岐数 B_{16} から必要ラウンド数を見積もり，暗号全体の実装コストに基づいて候補を絞り込む．

必要ラウンド数の見積もり

設計段階の簡易評価として，2 ラウンドごとに分岐数 B_{16} に相当するアクティブ S-box が得られると見積もる．このとき，R ラウンドのアクティブ S-box 数を

$$A_{B_{16}}(R) = \left\lfloor \frac{R}{2} \right\rfloor B_{16} + (R \bmod 2)$$

とする．したがって，S-box 候補 $S_\tau^\star$ に対して必要な最小ラウンド数を

$$R_{B_{16},\tau} = \min\{R \mid A_{B_{16}}(R) \geq N_\tau\}$$

と定める．

実装コストに基づく候補絞り込み

S-box を 8 セルに適用する際には，各セルあたり τ サイクルに加えて，全体で 8 サイクルのパッキング処理が必要となる．さらに，ラウンド tweakey 加算を 9 サイクルと見積もると，線形層を除く 1 ラウンドあたりの基本コストは

$$C_{\mathrm{base}}(\tau) = 8(\tau + 1) + 9$$

表 **A·5** 分岐数に基づく絞り込み後の S-box 候補．

	基本コスト最小候補			少ラウンド候補		
B_{16}	τ	R	$C_{B_{16},\tau}$	τ	R	$C_{B_{16},\tau}$
1	8	27	2187	9	25	2225
2	8	14	1134	9	13	1157
3	9	9	801	–	–	–
4	9	7	623	–	–	–
5	8	6	486	–	–	–
6	7	6	438	9	5	445
7	8	4	324	–	–	–
8	8	4	324	–	–	–
9	7	4	292	–	–	–

となる．したがって，$R_{B_{16},\tau}$ ラウンド分の基本コストを

$$C_{B_{16},\tau} = R_{B_{16},\tau} C_{\mathrm{base}}(\tau)$$

とする．

線形層 1 回あたりの実装コストを C_M とすると，最終ラウンドでは線形層を省略するため，暗号全体の推定コストは

$$C_{B_{16},\tau}^{\mathrm{total}} = C_{B_{16},\tau} + (R_{B_{16},\tau} - 1) C_M$$

となる．ただし，C_M は分岐数 B_{16} を実現する線形層に依存するため，この段階では最適な S-box 候補を一意に決定できない．

そこで，各分岐数 B_{16} について，別の候補より $R_{B_{16},\tau}$ が大きく，かつ $C_{B_{16},\tau}$ も大きい候補を除外する．このような候補は，任意の $C_M \geq 0$ に対して総コストが小さくなることがないためである．絞り込み後の候補を表 A·5 に示す．

この段階では線形層の実装コスト C_M が未定であるため，残った候補間の優劣は確定しない．

A.2.3 再帰型線形層の探索と最終選択

前節で残した S-box 候補に対して再帰型線形層を探索し，実現可能な分岐数 B_{16} と実装コスト C_M を求める．その後，各組合せの総コストを比較し，最終的な S-box と線形層を選択する．

再帰型線形層の探索

線形層を構成する基本演算を

$$X_d \leftarrow X_d \oplus (X_s \lll_{32} k), \qquad d \neq s$$

とする．更新先 d は 4 通り，参照元 s は 3 通り，回転量 k は 32 通りであるため，1 回の更新には $4 \cdot 3 \cdot 32 = 384$ 通りの候補が存在する．したがって，m 更新からなる線形層の探索空間は 384^m となり，6 更新でも

$$384^6 \simeq 3.2 \times 10^{15}$$

に達する．制約なしでは 6 更新までを全探索したが，得られた最大分岐数は $B_{16} = 4$ であり，それ以上の更新数を同様に探索することは非現実的である．

© 2026 Information Processing Society of Japan

そこで，4 更新を 1 段とするステージ制を導入する．各段では更新先の順序を $(0,1,2,3)$ に固定し，4 つの 32 ビットワードをそれぞれ 1 回ずつ参照元として用いる．$d \neq s$ より，参照元の選択は固定点を持たない 4 要素の置換 9 通りである．さらに，段内では共通の回転量 k を用いるため，1 段あたり

$$9 \cdot 32 = 288$$

通りとなる．2 段，3 段，4 段ではそれぞれ

$$288^2 = 82{,}944, \qquad 288^3 = 23{,}887{,}872,$$

$$288^4 = 6{,}879{,}707{,}136$$

通りを全探索した．結果を表 A·6 に示す．

表 A·6 再帰型線形層の探索結果．

探索方法	C_M	段数	最大 B_{16}
制約なしの探索	6	–	4
ステージ制	8	2	6
ステージ制	12	3	7
ステージ制	16	4	8

これにより，各分岐数 B_{16} を実現する線形層の実装コスト C_M が得られる．

最終パラメータの選択

得られた (B_{16}, C_M) を前節の S-box 候補と組み合わせ，総コスト $C^{\text{total}}_{B_{16},\tau}$ を比較する．各線形層について総コストを最小化する組合せを表 A·7 に示す．

表 A·7 S-box と線形層の組合せによる推定コストの比較．

B_{16}	C_M	τ	R	$C^{\text{total}}_{B_{16},\tau}$
4	6	9	7	659
6	8	9	5	477
7	12	8	4	360
8	16	8	4	372

最小となるのは，$B_{16} = 7$，$C_M = 12$ の線形層と $\tau = 8$ の S-box の組合せである．このとき $R = 4$，$C_{\text{base}}(8) = 81$ であり，

$$C^{\text{total}}_{7,8} = 4 \cdot 81 + (4-1) \cdot 12 = 360$$

を得る．

最後に，3 段の探索で得られた 12 更新かつ $B_{16} = 7$ を満たす，具体的な線形層を選択する．同様の候補は 9,700 候補から得られた．この内, 回転量 0 となる段数と非零のバイト境界回転

$$k \in \{8, 16, 24\}$$

を持つ段を最大化すると，176 候補が残った．これらを区別する追加の安全性・性能上の指標は得られなかったため，条件を満たす候補の一つとして回転量

$$(24, 0, 13)$$

を持つ線形層を採用した．

A.3 その他の安全性評価

本付録では，本文で省略したその他の安全性評価のうち，紙面の都合上，乗算差分攻撃，代数攻撃，および積分攻撃に対する評価を示す．

A.3.1 乗算差分攻撃

乗算差分攻撃は，補数型関係や 2 倍関係など，法乗算と整合する入出力関係を利用する．ここでは，データ処理部の非線形性が集中する 16 ビット S-box S_{16} に対して，このような関係を評価する．

補数型関係について，全 2^{16} 入力を全数検査した結果，

$$2^{-16} \left|\{x :\ S_{16}(x \oplus \texttt{0xffff}) = S_{16}(x) \oplus \texttt{0xffff}\}\right| = 2^{-15}$$

を得た．また，乗算差分攻撃で典型的に考慮される 2 倍関係について，

$$2^{-16} \left|\{x :\ S_{16}(2x) = 2S_{16}(x)\}\right| = \frac{4108}{2^{16}} \approx 2^{-4.00}$$

を得た．ここで，2 倍は法 2^{16} で計算する．S_{16}^{-1} に対しても同じ値が得られた．

定数乗算 M_a は 2 倍関係と可換である一方，S_{16} 内部のシフト–XOR 写像および回転–XOR 写像は，この関係を決定的には保存しない．すべてのセルがアクティブな S-box 層を通してセル単位の 2 倍関係を維持するには，各 16 ビットセルが局所条件

$$S_{16}(2x) = 2S_{16}(x)$$

を満たす必要があり，その確率は 1 セル当たり $4108/2^{16}$ である．さらに，線形層 $\mathcal{M}$ は XOR と回転によってワードを混合するため，セル単位の乗算関係を保存しない．任意のアクティブな関係は 1.5 ラウンド後に 8 個すべての 16 ビット S-box へ拡散する．したがって，乗算差分関係を複数ラウンドにわたって決定的に伝播させることはできず，提案方式に対する有効な乗算差分攻撃は確認されていない．

A.3.2 代数攻撃

$\mathbb{F}_2$ 上でラウンド関数の代数次数を追跡することにより，代数攻撃に対する耐性を評価する．XOR 演算および線形層 $\mathcal{M}$ は線形であるため，代数次数を増加させない．次数の増加は，16 ビット非線形置換 S_{16} とホワイトニング処理の法加算によって生じる．

S_{16} のすべての Boolean 成分について ANF を計算した．任意の非零マスク $v \in \mathbb{F}_2^{16}$ に対して，成分関数 $v \cdot S_{16}$ の代数次数は 15 であった．S_{16}^{-1} についても同じ性質が成り立つ．したがって，アクティブな 16 ビットセルは，S_{16} を

© 2026 Information Processing Society of Japan

1 回適用した時点で局所的に可能な最大次数へ到達する．

この局所次数をラウンド関数全体へ伝播させると，1 ラウンド後のアクティブセルの次数は 15 となる．さらに，1.5 ラウンド完全拡散により，アクティビティは 8 個すべての 16 ビット S-box へ到達する．次数伝播解析では，2 ラウンド後に次数 127 へ到達した．これは 128 ビット置換の成分関数が取り得る最大の代数次数であり，以降のラウンドでも最大次数の状態が維持される．

256 ビット鍵方式も同じデータ処理部を用いる．固定されたマスター鍵のもとでは，交互に用いる部分鍵は XOR でのみ注入されるため，上記の次数解析をそのまま適用できる．したがって，4 方式の代数表現はいずれも，128 ビット状態全体にわたる高次数方程式に加えて，tweakey 更新およびホワイトニング処理による依存関係を含む．Gröbner 基底法や SAT 解法を含め，全数探索より効率的な直接的代数攻撃は確認されていない．

A.3.3 積分攻撃

積分攻撃は，平文集合の総和が秘密鍵に依存しない性質を利用する．この種の識別器は，代数次数の増加および division property の伝播と密接に関係する．前節で示したように，16 ビット S-box S_{16} の任意の非零成分関数は代数次数 15 を持ち，次数伝播解析では 2 ラウンド後に，128 ビット置換の成分関数が取り得る最大次数 127 へ到達する．この急速な次数増加により，データ処理部を通して低次数の積分構造を維持することは困難である．

さらに，ホワイトニング処理では XOR ではなく，32 ビットワードごとの法加算によって鍵を混合する．この設計は，法加算鍵ホワイトニングを用いるブロック暗号の積分耐性に関する枠組み [26] に着想を得ている．すべての方式で 128 ビット状態を四つの 32 ビットワードに分割し，マスター鍵から導出したホワイトニング値を $\boxplus_{32\times 4}$ によって入力および出力へ加算する．128 ビット鍵方式ではマスター鍵とその 64 ビット回転を用い，256 ビット鍵方式では二つの 128 ビット部分鍵を組み合わせてホワイトニング値を生成する．したがって，ホワイトニング処理は，データ処理部の前後に配置されたワード単位の法加算型 pre/post-whitening とみなせる．XOR ホワイトニングが鍵を線形に加えるのに対し，各 32 ビットワード内のキャリー伝播は，ANF に非線形な鍵依存項を導入する．

法加算鍵ホワイトニングの枠組みでは，独立なホワイトニング鍵を用いる t ワード単位の pre/post-whitening 構成について，選択した高次数 ANF 係数の独立性を検査する問題へ積分耐性を帰着できる．本方式に対応するワード分割は $t = 4$，ワード幅は $s = 32$ である．ただし，本方式ではホワイトニング値とラウンド tweakey が同一のマスター鍵から導出されるため，この形式的結果を直接適用することはできない．それでも，キャリーによって非線形な鍵依存項を導入する同じ機構が，本方式で法加算型ホワイトニングを採用する根拠となっている．

以上より，データ処理部の高い代数次数と，法加算型ホワイトニングが導入する非線形な鍵依存性の組合せにより，暗号全体を通して鍵に依存しない積分特性を維持することは困難である．4 方式のいずれに対しても，有効な積分識別器または積分鍵回復攻撃は確認されていない．

© 2026 Information Processing Society of Japan